

MODULE 1 – AUTOMATION WITH SELENIUM

EXPERIMENT 01 – MULTI-LOCATOR CHALLENGE

1. Module Name and Experiment Title

Module Name: Module 1 – Automation with Selenium

Experiment Number: 01

Experiment Title: Multi-Locator Challenge

Programming Language: Python

Automation Tool: Selenium WebDriver

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

Automate the login functionality of a web application using Selenium WebDriver by locating different web elements using different locator strategies.

The experiment requires identifying the **username field using `By.ID`**, the **password field using `By.NAME`**, and the **login button using `By.XPATH`**.

After entering the required login credentials and clicking the login button, verify that the user is successfully redirected to the expected inventory page by checking that the resulting URL contains `/inventory.html`.

3. Objective

The main objectives of this experiment are:

1. To understand Selenium WebDriver's element location mechanism.
2. To practice using multiple Selenium locator strategies.
3. To locate web elements using `By.ID`, `By.NAME`, and `By.XPATH`.
4. To automate a basic login workflow.
5. To verify successful navigation after login using URL validation.
6. To observe the automation process clearly by introducing a 3-second delay between major steps.

7. To maximize the browser window during automation for better visibility.
-

4. Tools, Software and Concepts Used

Tools and Software

- Python
- Selenium WebDriver
- Google Chrome
- ChromeDriver
- Visual Studio Code

Concepts Used

- Selenium WebDriver
 - Web element identification
 - `By.ID`
 - `By.NAME`
 - `By.XPATH`
 - `find_element()`
 - `send_keys()`
 - `click()`
 - `time.sleep()`
 - Browser window maximization
 - URL verification
 - Browser automation
-

5. Source Code / Implementation Steps

Implementation Steps

1. Import the required Selenium and Python libraries.
2. Initialize the Chrome WebDriver.
3. Wait for 3 seconds for better visibility.
4. Maximize the browser window.
5. Wait for 3 seconds.
6. Open the SauceDemo web application.
7. Locate the username field using `By.ID`.
8. Enter the username and wait for 3 seconds.
9. Locate the password field using `By.NAME`.
10. Enter the password and wait for 3 seconds.
11. Locate the login button using `By.XPATH`.
12. Click the login button.
13. Wait for 3 seconds to allow the page to load.
14. Retrieve the current browser URL.

15. Verify that the URL contains /inventory.html.
16. Display the login success message and current URL in the console.
17. Wait for 3 seconds.
18. Close the browser.

Note: A 3-second delay is introduced between the major automation steps to make the execution clearly visible during the demonstration.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Initialize Chrome WebDriver
driver = webdriver.Chrome()

time.sleep(3)

# Maximize browser window
driver.maximize_window()

time.sleep(3)

# Open the web application
driver.get("https://www.saucedemo.com/")

time.sleep(3)

# Enter username using ID locator
username = driver.find_element(By.ID, "user-name")

time.sleep(3)

username.send_keys("standard_user")

time.sleep(3)

# Enter password using NAME locator
password = driver.find_element(By.NAME, "password")

time.sleep(3)

password.send_keys("secret_sauce")

time.sleep(3)

# Locate login button using XPATH locator
login_button = driver.find_element(
    By.XPATH, "//input[@type='submit']"
)

time.sleep(3)

# Click login button
login_button.click()

time.sleep(3)
```

```
# Verify successful login
current_url = driver.current_url

time.sleep(3)

assert "/inventory.html" in current_url

print("Login successful!")
print("Current URL:", current_url)

time.sleep(3)

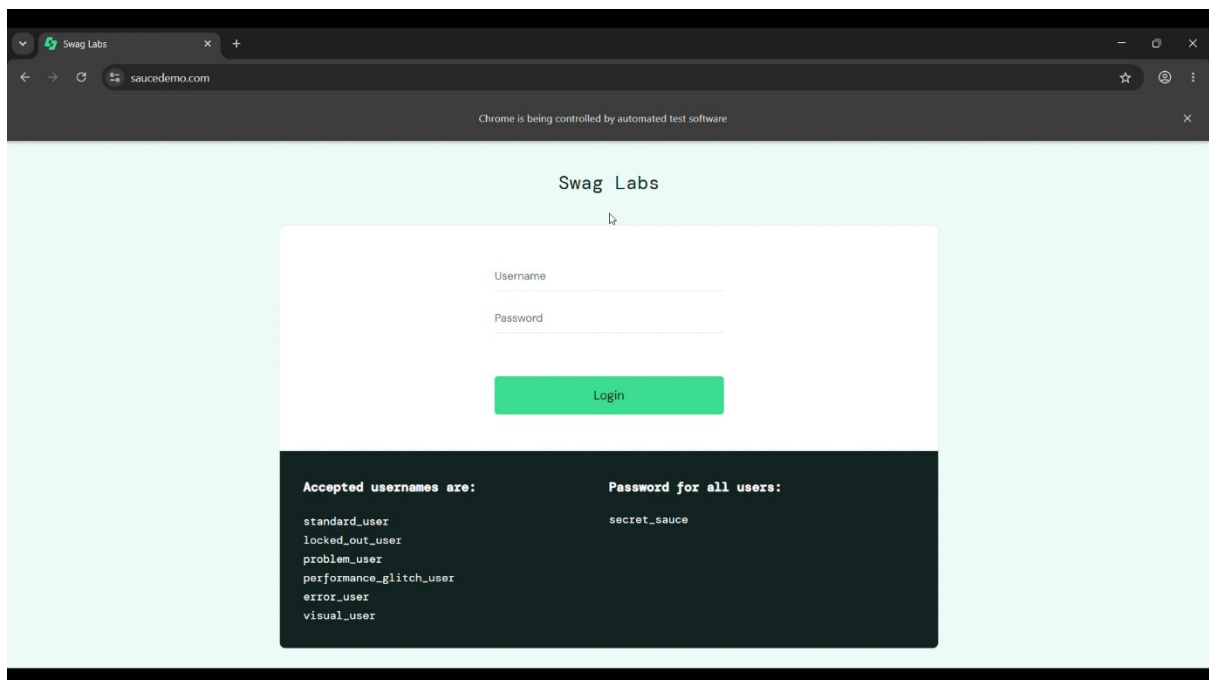
# Close the browser
driver.quit()
```

6. Output with Clearly Labelled Screenshots

The Selenium automation script was executed successfully on the SauceDemo website. The following screenshots demonstrate the login process, from opening the application to verifying successful navigation to the inventory page.

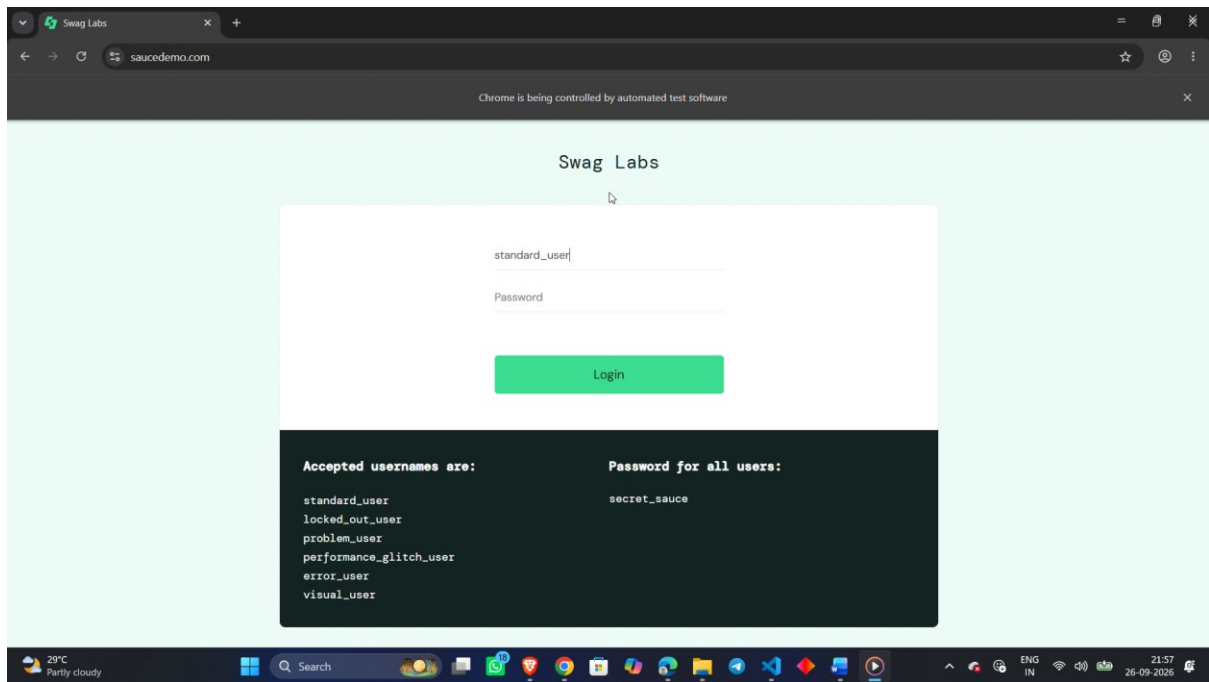
Screenshot 1: Browser Maximized and Login Page

Description: The Chrome browser is maximized, and the SauceDemo login page is displayed before entering the login credentials.



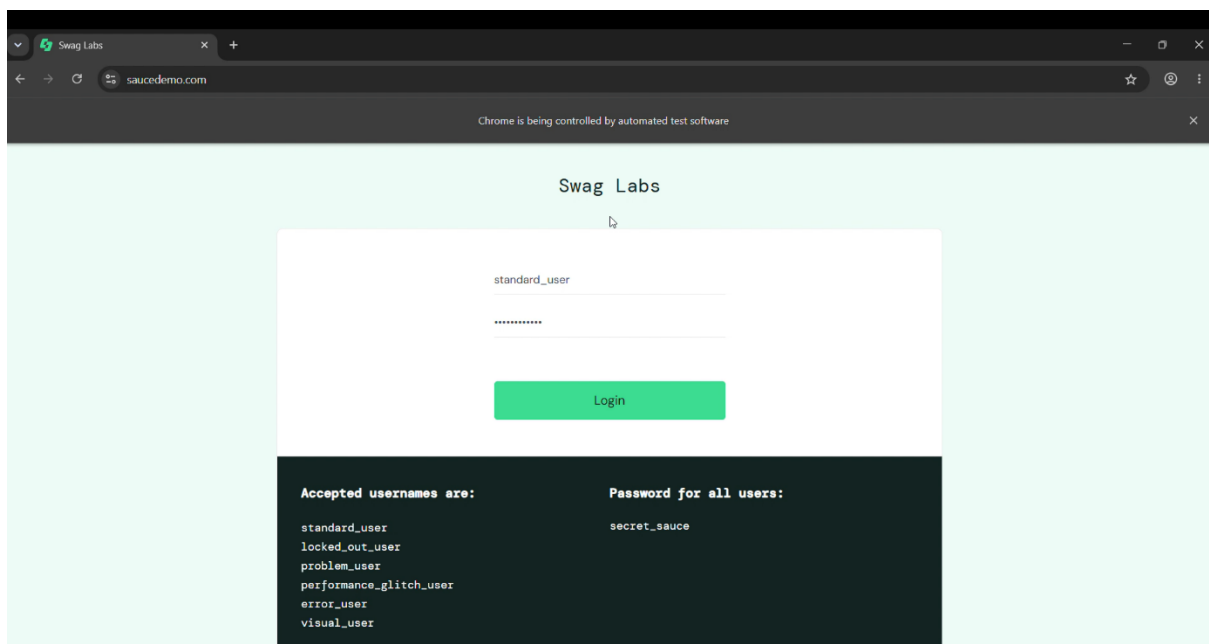
Screenshot 2: Username Entered

Description: The username field is located using the `By.ID` locator, and the username `standard_user` is entered successfully.



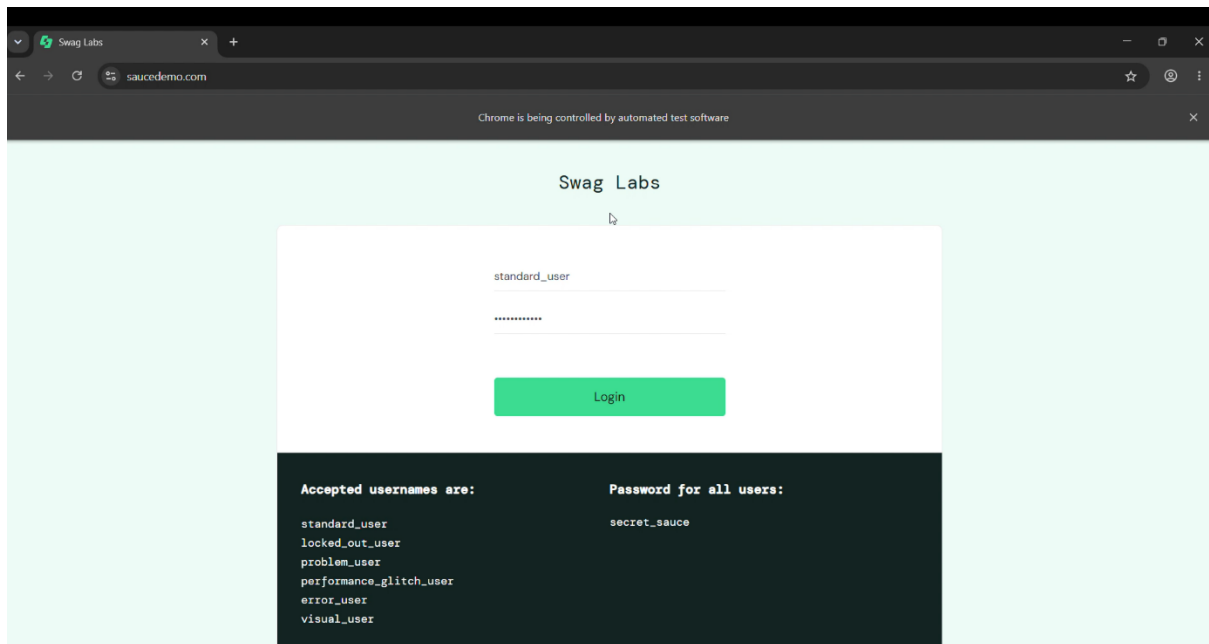
Screenshot 3: Password Entered

Description: The password field is located using the `By.NAME` locator, and the password `secret_sauce` is entered successfully.



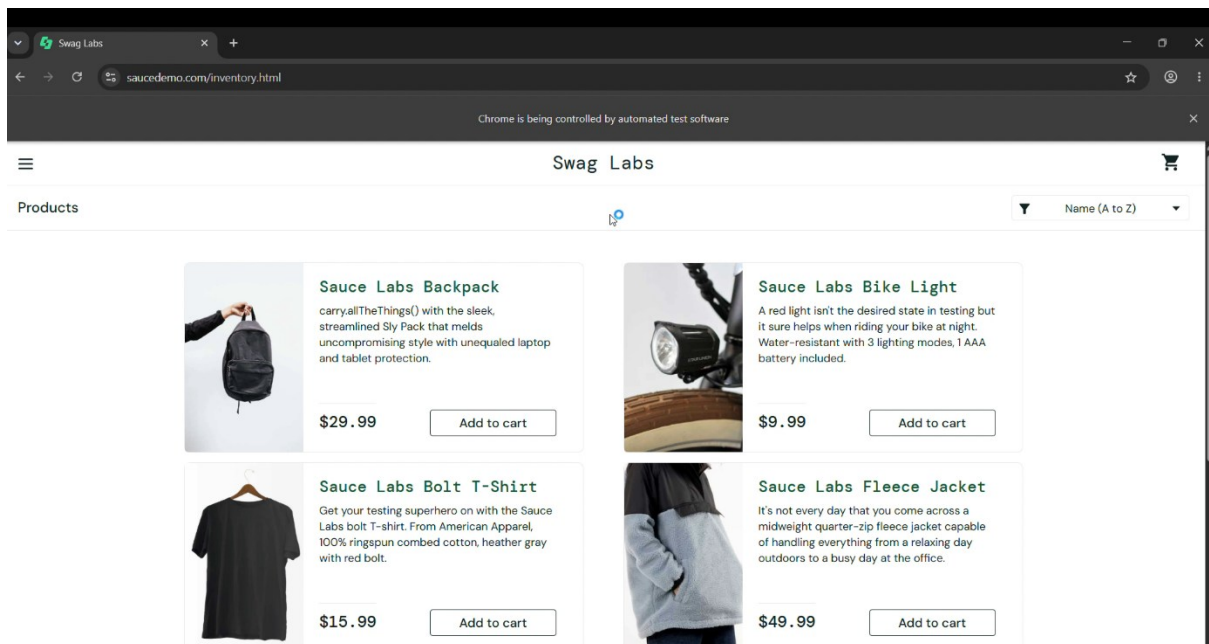
Screenshot 4: Login Action

Description: The login button is located using the `By.XPATH` locator and clicked to submit the login form.



Screenshot 5: Successful Login / Inventory Page

Description: After successful authentication, the application redirects to the inventory page. The browser URL contains `/inventory.html`, confirming successful navigation.



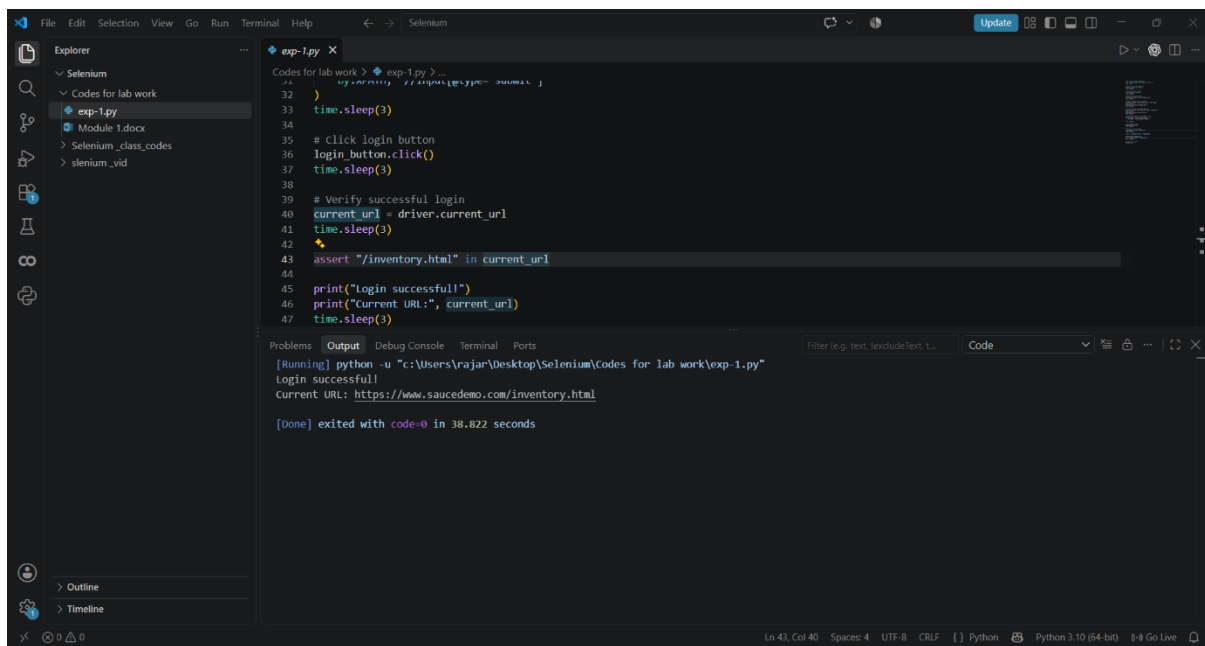
Screenshot 6: Console Output

Description: The VS Code terminal displays the successful login message and the current URL after completing the automation.

Console Output:

```
Login successful!
```

Current URL: <https://www.saucedemo.com/inventory.html>



The screenshot shows a Visual Studio Code editor window with a Selenium Python script named `exp-1.py` open. The script performs a login on the saucedemo.com website and verifies the current URL. The output window at the bottom shows the successful execution of the script.

```
exp-1.py
32
33 time.sleep(3)
34
35 # click login button
36 login_button.click()
37 time.sleep(3)
38
39 # Verify successful login
40 current_url = driver.current_url
41 time.sleep(3)
42
43 assert "/inventory.html" in current_url
44
45 print("Login successful!")
46 print("Current URL:", current_url)
47 time.sleep(3)
```

Output

```
[Running] python -u "c:\Users\rajar\Desktop\Selenium\Codes for lab work\exp-1.py"
Login successful!
Current URL: https://www.saucedemo.com/inventory.html

[Done] exited with code=0 in 38.822 seconds
```

7. Result, Observation and Conclusion

Result

The Selenium WebDriver script successfully automated the login process using multiple locator strategies.

The username, password, and login button were located using `By.ID`, `By.NAME`, and `By.XPATH`, respectively.

The successful login was verified by checking that the current URL contained `/inventory.html`. The script displayed the login success message and the current URL in the console.

Observation

Different Selenium locator strategies can be used to identify and interact with web elements.

In this experiment:

- `By.ID` was used to locate the username field.
- `By.NAME` was used to locate the password field.
- `By.XPATH` was used to locate the login button.
- `send_keys()` was used to enter the login credentials.
- `click()` was used to submit the login form.
- URL verification was used to confirm successful navigation.

A 3-second delay between major steps made the browser actions easier to observe during the demonstration. The browser was also maximized to improve visibility.

Conclusion

The experiment successfully demonstrated the use of multiple Selenium locator strategies for web automation.

It also provided practical experience in performing user interactions, maximizing the browser window, introducing execution delays for demonstration purposes, and verifying the result of an automated login operation.

The successful execution of the script confirmed that Selenium WebDriver can automate a basic login workflow and validate the resulting navigation.